

PYTHON FOR DATA

```
import pandas as pd
import matplotlib.pyplot as plt

data = pd.read_csv('data.csv')

df.describe()

plt.plot()
data.drop('date', axis=1)

for row in range(len(data)):
    df.loc[row, 'date'] = ...
```



**THE COMPLETE GUIDE TO
DATA ANALYSIS, MANIPULATION,
AND VISUALIZATION**

Disclaimer

This eBook is intended solely for educational and informational purposes. Every effort has been made to ensure that the information presented is accurate, relevant, and up to date at the time of publication. However, the field of Python programming, data analysis, data visualization, artificial intelligence, and machine learning evolves rapidly. New libraries, frameworks, language features, and best practices may emerge after this publication.

The examples, code snippets, explanations, workflows, and techniques provided in this book are designed to help readers understand Python programming concepts and modern data-related technologies. While every attempt has been made to verify the correctness of the code and explanations, no guarantee is provided that all examples will work without modification in every software environment, operating system, or future Python version.

Readers are encouraged to verify important information through the official documentation of Python and the respective libraries before using the code in production environments. Practical implementation may require adjustments depending on software versions, project requirements, hardware configurations, and third-party dependencies.

The author and publisher assume no responsibility for any direct or indirect loss, damage, technical issue, financial loss, or data loss resulting from the use or misuse of the information contained in this publication. Users are solely responsible for testing, validating, and securing their own applications, scripts, datasets, and production systems.

Any trademarks, product names, software names, company names, or brand names mentioned in this book are the property of their respective owners and are used only for identification and educational purposes. Their inclusion does not imply any endorsement, sponsorship, or affiliation.

This publication does not provide professional legal, financial, cybersecurity, or business advice. Readers should seek appropriate professional guidance whenever required.

No part of this publication may be reproduced, copied, stored in a retrieval system, transmitted, translated, distributed, or published in any form or by any means — electronic, mechanical, photocopying, recording, scanning, or otherwise — without prior written permission from the copyright holder, except for brief quotations used in reviews or academic references as permitted by applicable copyright laws.

By reading and using this eBook, you acknowledge that you understand and accept the terms of this disclaimer and agree to use the information responsibly and at your own discretion.

Preface

A PRACTICAL PATH FROM PYTHON BASICS TO DATA WORK

Python for Data is designed to connect programming fundamentals with the way data work is actually performed. The goal is not to memorize syntax, but to build enough understanding to read data, transform it, investigate patterns, and communicate useful results.

The book moves from Python foundations into NumPy, pandas, data cleaning, visualization, exploratory analysis, and machine learning. Each stage builds on the previous one so that readers can understand not only what a tool does, but when and why it should be used.

Examples are intentionally practical. Readers are encouraged to type the code, change inputs, inspect outputs, try unfamiliar datasets, and treat errors as part of the learning process. The most useful skill is the ability to turn a vague data question into a clear, testable workflow.

The Python ecosystem changes quickly. Library APIs, recommended workflows, and best practices should therefore be checked against current official documentation when this book is used for real projects.

Use this book as both a learning guide and a working reference. Build small projects, keep your code readable, document assumptions, and validate conclusions before treating them as facts.

LEARNING PRINCIPLE

Learn → Run → Inspect → Change → Repeat

The fastest route to confidence is active practice with small, understandable examples.

Table of Contents

PART 1 • FOUNDATIONS, NUMPY, AND PANDAS

Introduction to Python for Data Science	6
Why Python for Data?	6
Setting Up Your Environment	6
The Core Toolkit	6
How This Book Is Organized	7
Getting Started with NumPy	8
Why Not Just Use Python Lists?	8
Creating Arrays	8
Indexing and Slicing	8
Vectorized Operations	9
NumPy Arrays: Operations and Broadcasting	10
Broadcasting Rules	10
Aggregations Along an Axis	10
Reshaping and Stacking	10
Random Numbers and Reproducibility	11
Introduction to Pandas	12
Series and DataFrame	12
Inspecting a DataFrame	12
Selecting Data	12
Adding and Modifying Columns	13
Reading and Writing Data	13
Data Cleaning and Manipulation with Pandas	14
Handling Missing Values	14
Removing Duplicates and Fixing Types	14
Grouping and Aggregating	14
Merging and Joining	14
Reshaping with Pivot Tables	15
Chaining Operations	15

Table of Contents

PART 2 • VISUALIZATION, EDA, MACHINE LEARNING, AND REFERENCE

Data Visualization with Matplotlib	16
The Anatomy of a Matplotlib Figure	16
Common Chart Types	16
Multi-Panel Figures	16
Plotting Directly from pandas	17
Exploratory Data Analysis	18
A Repeatable EDA Checklist	18
Summary Statistics and Correlation	18
Spotting Outliers	18
From EDA to Feature Ideas	18
Introduction to Machine Learning	20
Supervised vs. Unsupervised Learning	20
Regression vs. Classification	20
The Train/Test Split	20
Overfitting and Underfitting	20
Building Your First ML Model with Scikit-learn	21
The Fit / Predict Pattern	21
A Complete Classification Example	21
Preprocessing with Pipelines	21
Feature Importance	22
Model Evaluation and Next Steps	23
Regression Metrics	23
Classification Metrics	23
Cross-Validation	23
Where to Go from Here	23
Appendix — Quick Reference Cheat Sheet	25
About the Author	26
Index	27

CHAPTER 1

Introduction to Python for Data Science

Python has become the de facto language of data science, and for good reason. Its clean, readable syntax lowers the barrier to entry, while a mature ecosystem of libraries makes it possible to go from raw data to a trained machine learning model without ever leaving the language. This chapter sets the stage for everything that follows: why Python won the data science ecosystem war, how to set up a productive environment, and the small set of tools you will use in nearly every project.

Why Python for Data?

Three qualities explain Python's dominance in data work. First, readability: Python code reads almost like pseudocode, which means the gap between an idea and working code is small. Second, a mature scientific stack: libraries such as NumPy, pandas, Matplotlib, and scikit-learn have been refined over more than a decade and are battle-tested in production. Third, community: when you get stuck, the odds are overwhelming that someone has already solved your exact problem and written about it.

Setting Up Your Environment

Most data professionals use a scientific Python distribution rather than a bare installation, because it bundles the compiled dependencies that libraries like NumPy rely on. A minimal, reliable setup looks like this:

```
# Create an isolated environment
python -m venv data-env
source data-env/bin/activate # On Windows: data-env\Scripts\activate

# Install the core stack
pip install numpy pandas matplotlib scikit-learn jupyterlab
```

The Core Toolkit

- **NumPy** — fast, memory-efficient arrays and the numerical engine underneath almost everything else.
- **pandas** — labeled, tabular data structures (DataFrames) built for cleaning and analysis.
- **Matplotlib** — the foundational plotting library; most other Python charting tools build on it.
- **scikit-learn** — a consistent, well-documented API for classical machine learning.
- **Jupyter** — an interactive notebook environment ideal for exploratory work.

How This Book Is Organized

Chapters 2 and 3 build your numerical foundation with NumPy. Chapters 4 through 7 cover pandas and visualization in depth, since real-world data is almost always messy tabular data before it is anything else. Chapters 8 through 10 introduce machine learning with scikit-learn, treating it as the natural extension of the data manipulation skills you will already have. Each chapter is self-contained but builds on the last, and every concept is paired with runnable code.

A Note on the Examples

Every code listing in this book is written to run top-to-bottom in a single script or notebook cell. Where a listing depends on a variable from an earlier example, that dependency is called out explicitly.

PRACTICE NOTE

Change the array shape and inspect the result. Compare a vectorized operation with an equivalent Python loop.

CHAPTER 2

Getting Started with NumPy

NumPy — short for Numerical Python — provides the `ndarray`, a fast, fixed-type array that underlies pandas, scikit-learn, and most of the scientific Python ecosystem. Understanding arrays deeply pays off everywhere else in this book.

Why Not Just Use Python Lists?

Python lists are flexible but slow for numerical work: each element is a full Python object with its own type information and memory overhead. A NumPy array stores elements of a single, fixed type in one contiguous block of memory, which allows operations to run as compiled, vectorized loops rather than interpreted Python loops — often 10 to 100 times faster.

Creating Arrays

```
import numpy as np

a = np.array([1, 2, 3, 4, 5])
b = np.zeros((3, 4))          # 3x4 array of zeros
c = np.ones((2, 2))           # 2x2 array of ones
d = np.arange(0, 10, 2)        # [0 2 4 6 8]
e = np.linspace(0, 1, 5)       # 5 evenly spaced points from 0 to 1
f = np.random.rand(3, 3)       # 3x3 array of random floats in [0, 1)

print(a.shape, a.dtype)        # (5,) int64
```

Indexing and Slicing

NumPy slicing mirrors Python list slicing but extends naturally to multiple dimensions using comma-separated indices.

```
grid = np.arange(12).reshape(3, 4)
# array([[ 0,  1,  2,  3],
#        [ 4,  5,  6,  7],
#        [ 8,  9, 10, 11]])

grid[1, 2]          # 6          -> single element
grid[0, :]           # row 0
grid[:, 1]           # column 1
grid[0:2, 1:3]       # sub-matrix
grid[grid > 5]       # boolean mask -> [6 7 8 9 10 11]
```


Vectorized Operations

The single biggest habit shift when moving from plain Python to NumPy is replacing explicit loops with vectorized expressions.

```
x = np.array([1, 2, 3, 4])
y = np.array([10, 20, 30, 40])

x + y          # array([11, 22, 33, 44])
x * 2          # array([2, 4, 6, 8])
np.sqrt(x)     # element-wise square root
np.sum(x)      # 10
np.mean(y)     # 25.0
x.dot(y)       # dot product -> 300
```

Common Pitfall: Views vs. Copies

Slicing a NumPy array returns a view into the original data, not a copy. Modifying the slice modifies the original array. Call `.copy()` explicitly when you need an independent array.

PRACTICE NOTE

Change the array shape and inspect the result. Compare a vectorized operation with an equivalent Python loop.

CHAPTER 3

NumPy Arrays: Operations and Broadcasting

This chapter goes deeper into the mechanics that make NumPy fast and expressive: broadcasting, aggregation along axes, and reshaping. Mastering these ideas is what lets you write concise code that would otherwise require nested loops.

Broadcasting Rules

Broadcasting lets NumPy perform operations on arrays of different shapes without writing explicit loops or duplicating data. Two dimensions are compatible when they are equal, or when one of them is 1. NumPy compares shapes from the trailing dimension backward.

```
prices = np.array([19.99, 5.49, 12.00])
tax_rate = 1.08

total = prices * tax_rate      # scalar broadcasts to every element

matrix = np.ones((3, 3))
row = np.array([1, 2, 3])
matrix + row                  # row broadcasts across each of the 3 rows
```

Aggregations Along an Axis

Aggregation functions accept an axis argument that controls which dimension is collapsed: axis=0 collapses rows (operating down each column), and axis=1 collapses columns (operating across each row).

```
scores = np.array([[88, 92, 79],
                   [95, 85, 91],
                   [70, 88, 84]])

scores.mean(axis=0)    # average per column -> [84.33 88.33 84.67]
scores.mean(axis=1)    # average per row    -> [86.33 90.33 80.67]
scores.max()           # overall max -> 95
scores.argmax(axis=1)  # index of the max in each row
```

Reshaping and Stacking

```
flat = np.arange(6)
grid = flat.reshape(2, 3)
back = grid.flatten()

a = np.array([1, 2, 3])
b = np.array([4, 5, 6])
np.vstack([a, b])           # stack as rows
np.hstack([a, b])           # concatenate as one row
np.column_stack([a, b])     # stack as columns
```

Random Numbers and Reproducibility

Nearly every modeling workflow depends on randomness — from shuffling data to initializing models — which makes reproducibility essential.

```
rng = np.random.default_rng(seed=42)
sample = rng.normal(loc=0, scale=1, size=1000) # 1000 draws from a normal distribution
```

PRACTICE NOTE

Change the array shape and inspect the result. Compare a vectorized operation with an equivalent Python loop.

CHAPTER 4

Introduction to Pandas

If NumPy is the engine, pandas is the dashboard. It introduces the Series and DataFrame — labeled, heterogeneous data structures purpose-built for the messy, tabular data you will encounter in almost every real project.

Series and DataFrame

```
import pandas as pd

s = pd.Series([10, 20, 30], index=['a', 'b', 'c'])

data = pd.read_csv('data.csv')      # from a CSV file
df = pd.DataFrame({
    'name': ['Ana', 'Ben', 'Cara'],
    'score': [88, 92, 79],
    'city': ['Delhi', 'Lagos', 'Lima'],
})
```

Inspecting a DataFrame

```
df.head()          # first 5 rows
df.info()          # dtypes, non-null counts, memory usage
df.describe()      # summary statistics for numeric columns
df.shape           # (rows, columns)
df.columns         # column labels
```

Selecting Data

pandas offers two primary selection tools: `.loc` for label-based indexing and `.iloc` for position-based indexing.

```
df['score']         # a single column -> Series
df[['name', 'score']] # multiple columns -> DataFrame

df.loc[0, 'name']   # label-based: row label 0, column 'name'
df.iloc[0, 1]       # position-based: first row, second column

df[df['score'] > 85] # boolean filtering
```

Adding and Modifying Columns

```
df['passed'] = df['score'] >= 80
df['score_pct'] = df['score'] / 100
df.rename(columns={'city': 'location'}, inplace=True)
```

Reading and Writing Data

pandas supports dozens of formats out of the box, including CSV, Excel, JSON, SQL, and Parquet.

```
df.to_csv('output.csv', index=False)
df.to_excel('output.xlsx', sheet_name='Sheet1')
pd.read_json('records.json')
pd.read_sql('SELECT * FROM users', connection)
```



CHAPTER 5

Data Cleaning and Manipulation with Pandas

Real datasets are rarely clean. This chapter covers the recurring cleaning tasks — missing values, duplicates, type conversions — as well as the grouping and merging operations that let you reshape and combine data.

Handling Missing Values

```
df.isnull().sum()           # count missing values per column
df.dropna()                 # drop rows with any missing value
df.dropna(subset=['score']) # drop only where 'score' is missing
df.fillna(0)                # fill missing values with 0
df['score'].fillna(df['score'].mean(), inplace=True)
```

Removing Duplicates and Fixing Types

```
df.drop_duplicates(inplace=True)
df['score'] = df['score'].astype(float)
df['signup_date'] = pd.to_datetime(df['signup_date'])
```

Grouping and Aggregating

The split-apply-combine pattern, exposed through `groupby()`, is one of the most powerful tools in pandas for summarizing data.

```
df.groupby('city')['score'].mean()

df.groupby('city').agg(
    avg_score=('score', 'mean'),
    n_students=('name', 'count')
```

PRACTICE NOTE

Load a small CSV, inspect its columns, filter two rows, and verify the resulting data before transforming it.

```
orders = pd.DataFrame({'order_id': [1, 2], 'customer_id': [101, 102]})
customers = pd.DataFrame({'customer_id': [101, 102], 'name': ['Ana', 'Ben']})

merged = pd.merge(orders, customers, on='customer_id', how='left')
```

Reshaping with Pivot Tables

```
pivot = df.pivot_table(
    values='score', index='city', columns='passed', aggfunc='mean'
)
```

Chaining Operations

Method chaining keeps pipelines readable: wrap the chain in parentheses and put one operation per line so each transformation is easy to scan and debug.

```
clean = (
    df.dropna(subset=['score'])
    .assign(score_pct=lambda d: d['score'] / 100)
    .query('score_pct >= 0.8')
    .sort_values('score', ascending=False)
)
```


CHAPTER 6

Data Visualization with Matplotlib

A well-made chart often communicates a finding faster than a paragraph of text. This chapter covers the Matplotlib fundamentals you will reuse constantly, plus pandas' built-in convenience plotting.

The Anatomy of a Matplotlib Figure

Every Matplotlib plot has a Figure (the overall canvas) containing one or more Axes (individual plots). Understanding this hierarchy makes multi-panel figures far less confusing.

```
import matplotlib.pyplot as plt

fig, ax = plt.subplots(figsize=(6, 4))
ax.plot(x, y, label='Revenue')
ax.set_title('Monthly Revenue')
ax.set_xlabel('Month')
ax.set_ylabel('Revenue ($)')
ax.legend()

plt.savefig('revenue.png', dpi=150, bbox_inches='tight')
```

Common Chart Types

```
ax.plot(x, y)           # line chart
ax.bar(categories, values) # bar chart
ax.scatter(x, y, alpha=0.6) # scatter plot
ax.hist(data, bins=30)    # histogram
ax.boxplot(data)         # box plot
```

Multi-Panel Figures

```
fig, axes = plt.subplots(1, 2, figsize=(10, 4))
```

PRACTICE NOTE

Load a small CSV, inspect its columns, filter two rows, and verify the resulting data before transforming it.

Plotting Directly from pandas

For quick exploratory plots, DataFrames and Series expose a `.plot()` accessor that wraps Matplotlib.

```
df['score'].plot(kind='hist', bins=15, title='Score Distribution')  
df.groupby('city')['score'].mean().plot(kind='bar')
```

Design Tips for Honest Charts

Start bar-chart axes at zero, label your axes with units, and avoid 3D effects — they distort how readers perceive magnitude. A clear, boring chart communicates more than a decorative one.

PRACTICE NOTE

Load a small CSV, inspect its columns, filter two rows, and verify the resulting data before transforming it.

CHAPTER 7

Exploratory Data Analysis

Exploratory Data Analysis (EDA) is the disciplined process of getting to know a dataset before modeling it: understanding its shape, spotting outliers, and forming hypotheses. This chapter walks through a repeatable EDA workflow.

A Repeatable EDA Checklist

- Check the shape, dtypes, and a sample of rows.
- Quantify missing values and decide how to treat them.
- Look at summary statistics for every numeric column.
- Examine the distribution of the target variable, if there is one.
- Visualize relationships between features with scatter plots and correlation heatmaps.
- Flag outliers and decide whether they are errors or real, informative extremes.

Summary Statistics and Correlation

```
df.describe(include='all')
df.corr(numeric_only=True)

import seaborn as sns
sns.heatmap(df.corr(numeric_only=True), annot=True, cmap='coolwarm')
```

Spotting Outliers

The interquartile range (IQR) method is a simple, robust way to flag candidate outliers without assuming a normal distribution.

```
q1, q3 = df['score'].quantile([0.25, 0.75])
iqr = q3 - q1
lower, upper = q1 - 1.5 * iqr, q3 + 1.5 * iqr

outliers = df[(df['score'] < lower) | (df['score'] > upper)]
```

From EDA to Feature Ideas

Good EDA naturally produces a list of candidate features and cleaning steps for modeling: which columns need imputation, which categorical variables need encoding, and which numeric features are skewed enough to warrant a transform. Keep notes as you go — they become the backbone of your modeling plan in the next three chapters.

PRACTICE NOTE

Choose a chart that matches the question. Label axes, make units explicit, and remove decoration that does not add information.

CHAPTER 8

Introduction to Machine Learning

With clean, well-understood data in hand, we turn to machine learning: teaching a model to make predictions from examples rather than from hand-written rules. This chapter introduces the vocabulary and mental model you need before touching scikit-learn.

Supervised vs. Unsupervised Learning

In supervised learning, every training example has a known answer (a label), and the model learns to map inputs to that answer — for example, predicting house prices from square footage. In unsupervised learning, there are no labels, and the model instead looks for structure, such as clusters of similar customers.

Regression vs. Classification

- **Regression** predicts a continuous number, such as a price, a temperature, or a duration.
- **Classification** predicts a category, such as spam vs. not-spam, or which of five product tiers a customer will choose.

The Train/Test Split

A model that is only ever evaluated on the data it was trained on will look deceptively good. Splitting off a held-out test set lets you estimate how the model will perform on data it has never seen.

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

Overfitting and Underfitting

An overfit model memorizes noise in the training data and performs poorly on new data. An underfit model is too simple to capture the real pattern and performs poorly everywhere. Balancing this trade-off — often by tuning model complexity and using validation data — is the central challenge of applied machine learning.

CHAPTER 9

Building Your First ML Model with Scikit-learn

scikit-learn's greatest strength is consistency: nearly every model shares the same fit / predict interface, so once you learn the pattern once, you can apply it to dozens of algorithms. This chapter builds a complete, working model end to end.

The Fit / Predict Pattern

```
from sklearn.linear_model import LinearRegression

model = LinearRegression()
model.fit(X_train, y_train)          # learn from training data
predictions = model.predict(X_test)  # predict on new data
```

A Complete Classification Example

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

X = df[['age', 'income', 'tenure_months']]
y = df['churned']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42
)

clf = RandomForestClassifier(n_estimators=200, random_state=42)
clf.fit(X_train, y_train)

preds = clf.predict(X_test)
print('Accuracy:', accuracy_score(y_test, preds))
```

PRACTICE NOTE

Choose a chart that matches the question. Label axes, make units explicit, and remove decoration that does not add information.

bugs. accidentally fitting a scaler on the test set.

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

pipe = Pipeline([
    ('scaler', StandardScaler()),
    ('model', LogisticRegression()),
])

pipe.fit(X_train, y_train)
pipe.score(X_test, y_test)
```

Feature Importance

```
importances = pd.Series(clf.feature_importances_, index=X.columns)
importances.sort_values(ascending=False).plot(kind='bar')
```



CHAPTER 10

Model Evaluation and Next Steps

A model is only as trustworthy as its evaluation. This closing chapter covers the key metrics for regression and classification, cross-validation, and where to go from here.

Regression Metrics

```
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

mae = mean_absolute_error(y_test, predictions)
rmse = mean_squared_error(y_test, predictions, squared=False)
r2 = r2_score(y_test, predictions)
```

Classification Metrics

Accuracy alone can be misleading, especially with imbalanced classes. Precision, recall, and the F1 score, together with a confusion matrix, give a fuller picture.

```
from sklearn.metrics import classification_report, confusion_matrix

print(classification_report(y_test, preds))
print(confusion_matrix(y_test, preds))
```

Cross-Validation

A single train/test split can be noisy, especially with smaller datasets. K-fold cross-validation trains and evaluates the model K times on different splits and averages the results for a more reliable estimate.

```
from sklearn.model_selection import cross_val_score

scores = cross_val_score(clf, X, y, cv=5, scoring='accuracy')
print(scores.mean(), scores.std())
```

Where to Go from Here

- Practice on real, messy datasets from sources such as Kaggle or your own work.
- Learn feature engineering in depth — it typically matters more than model choice.

- Explore gradient boosting libraries (XGBoost, LightGBM) for tabular data.
- Study model interpretability tools such as SHAP once your models leave the notebook.
- Learn the basics of deploying a model behind a simple API.

Closing Thought

The tools in this book — NumPy, pandas, Matplotlib, and scikit-learn — cover perhaps 80% of day-to-day data work. The remaining 20% is judgment: knowing which question to ask, which metric actually matters for the business, and when a simple model beats a complicated one. That judgment only comes from practice. Go build something.

PRACTICE NOTE

Change the array shape and inspect the result. Compare a vectorized operation with an equivalent Python loop.

Appendix — Quick Reference Cheat Sheet

Task	Code
Read a CSV	<code>pd.read_csv('file.csv')</code>
Inspect data	<code>df.head(), df.info(), df.describe()</code>
Filter rows	<code>df[df['col'] > value]</code>
Select columns	<code>df[['a', 'b']]</code>
Handle missing data	<code>df.dropna(), df.fillna(value)</code>
Group and aggregate	<code>df.groupby('col').mean()</code>
Merge two tables	<code>pd.merge(df1, df2, on='key')</code>
Create an array	<code>np.array(...), np.zeros((r, c))</code>
Reshape an array	<code>arr.reshape(r, c)</code>
Basic plot	<code>plt.plot(x, y); plt.show()</code>
Train/test split	<code>train_test_split(X, y, test_size=0.2)</code>
Fit a model	<code>model.fit(X_train, y_train)</code>
Evaluate accuracy	<code>accuracy_score(y_test, preds)</code>

PRACTICE NOTE

Change the array shape and inspect the result. Compare a vectorized operation with an equivalent Python loop.

About the Author

Dr. Alex Chen is a data scientist and educator with over a decade of experience building data pipelines and machine learning systems across the finance and technology sectors. Alex holds a Ph.D. in Computer Science and has taught Python for data analysis to thousands of students, with a focus on turning abstract statistical ideas into concrete, runnable code.

This book grew out of that teaching experience: the questions that came up most often in workshops and office hours became its chapters, and the mistakes students made most often became its tips and warnings.

With contributions from Sam Smith.

PRACTICE NOTE

Choose a chart that matches the question. Label axes, make units explicit, and remove decoration that does not add information.

Index

A

Accuracy score, 9
Aggregation (NumPy), 3
Array creation, 2
Axis parameter, 3

B

Boolean filtering, 2, 4
Broadcasting, 3

C

Classification, 8, 9, 10
Confusion matrix, 10
Correlation, 7
Cross-validation, 10

D

DataFrame, 4
Duplicates, removing, 5

E

EDA workflow, 7
Environment setup, 1

F

Feature importance, 9
fillna(), 5

G

groupby(), 5

I

IQR method, 7
Indexing, NumPy, 2
Indexing, pandas (.loc/.iloc), 4

M

Matplotlib figures, 6
Merging DataFrames, 5
Missing values, 5
Model evaluation, 10

N

NumPy arrays, 2, 3

O

Overfitting, 8

P

Pipelines, scikit-learn, 9
Pivot tables, 5

R

Random Forest, 9
Regression, 8, 10
Reshaping arrays, 3

S

scikit-learn, 8, 9, 10
Series (pandas), 4
Supervised learning, 8

T

Train/test split, 8
TOC — see Table of Contents

U

Unsupervised learning, 8

V

Vectorization, 2
Visualization, 6

THANK YOU

Python for Data